

Start coding or generate with AI.

EDA 1.Dataset info 2.Data Cleaning 3.Data visualisation

age: The age of the patient. sex: Gender of the patient (0: female, 1: male). cp: Type of chest pain. trestbps: Resting blood pressure. chol: Serum cholesterol. fbs: Fasting blood sugar > 120 mg/dl. restecg: Resting electrocardiographic results. thalach: Maximum heart rate achieved. exang: Exercise induced angina. oldpeak: ST depression induced by exercise relative to rest

```
# Hear we import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Create a dataframe
df = pd.read_csv('heart.csv')
#print(df)
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

df.shape

(303, 14)

df.columns

Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'], dtype='object')

```
# To get detail info about dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null    int64
1   sex         303 non-null    int64
2   cp          303 non-null    int64
3   trestbps    303 non-null    int64
4   chol        303 non-null    int64
5   fbs         303 non-null    int64
6   restecg     303 non-null    int64
7   thalach     303 non-null    int64
8   exang       303 non-null    int64
9   oldpeak     303 non-null    float64
10  slope       303 non-null    int64
11  ca          303 non-null    int64
12  thal        303 non-null    int64
13  target      303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
# to get statistical information about dataframe
df.describe()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	s
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	0.326733	1.039604	1.395016
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794	1.161075	0.610129
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000	0.000000	1.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	0.000000	0.800000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	1.000000	1.600000	2.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000

```
# check missing values
df.isnull().sum()
```

```

    0
age    0
sex    0
cp     0
trestbps  0
chol   0
fbs    0
restecg  0
thalach  0
exang   0
oldpeak  0
slope   0
ca      0
thal    0
target  0

dtype: int64
```

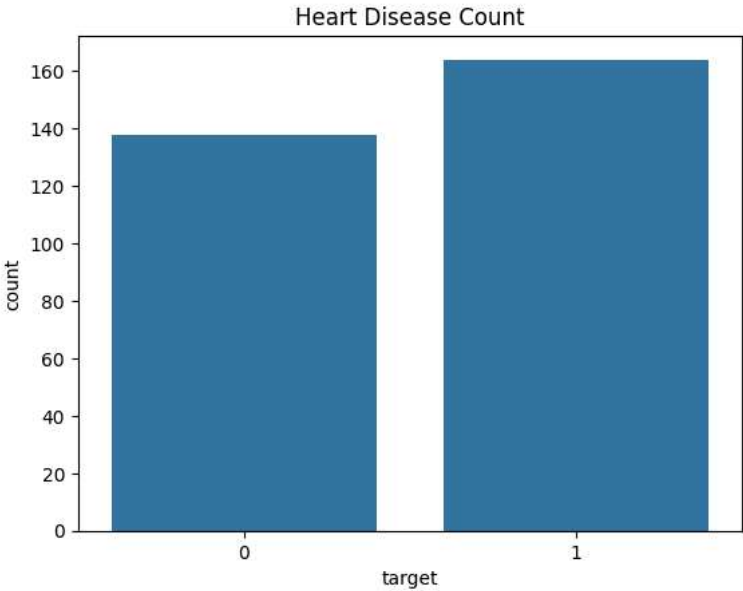
```
# check duplicate data items
print(df.duplicated().sum())
```

1

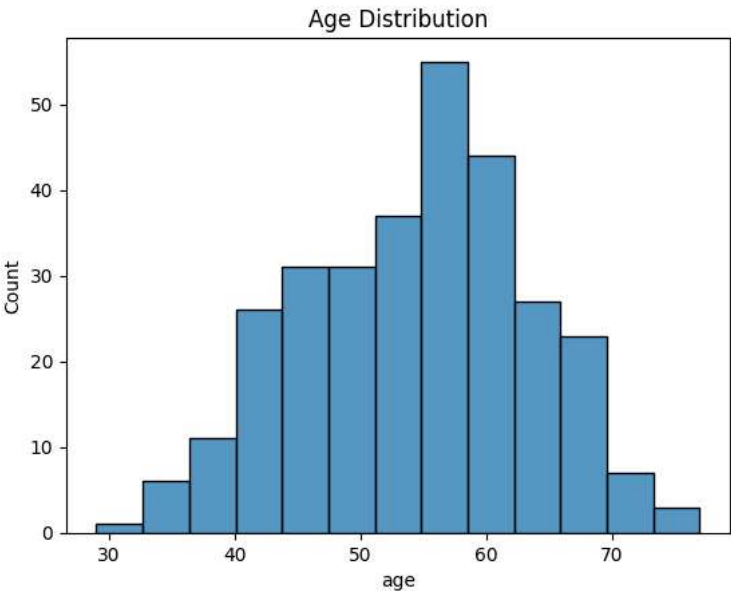
```
# Drop duplicates
df.drop_duplicates(inplace=True)
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
# Data visualization
# Target analysis
sns.countplot(x='target',data=df)
plt.title('Heart Disease Count')
plt.show()
```



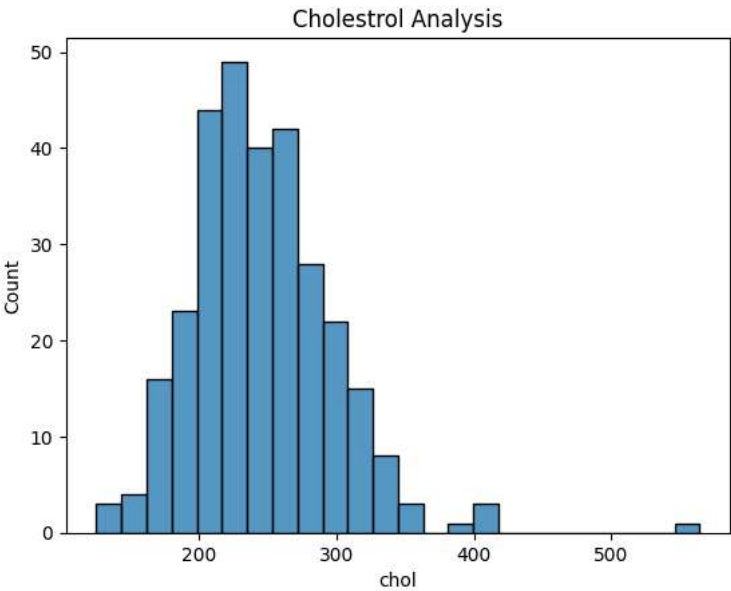
```
# Age Distribution
sns.histplot(df['age'])
plt.title('Age Distribution')
plt.show()
```



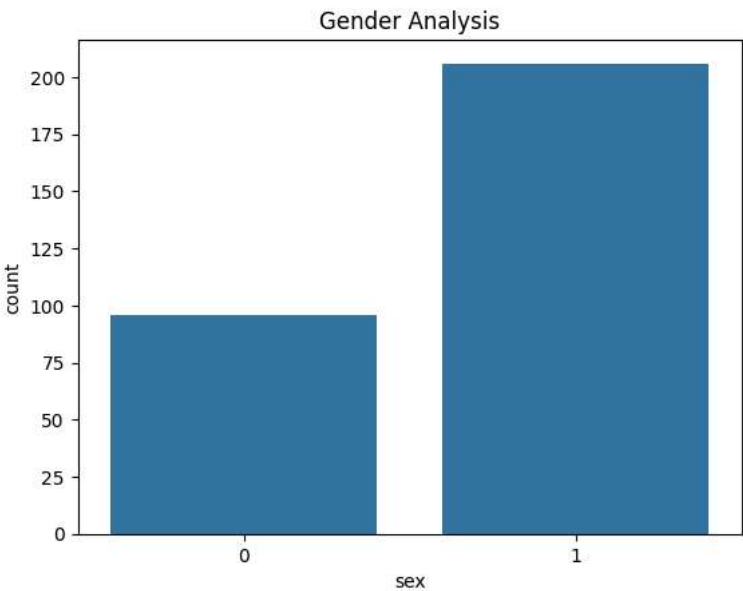
```
df.columns

Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

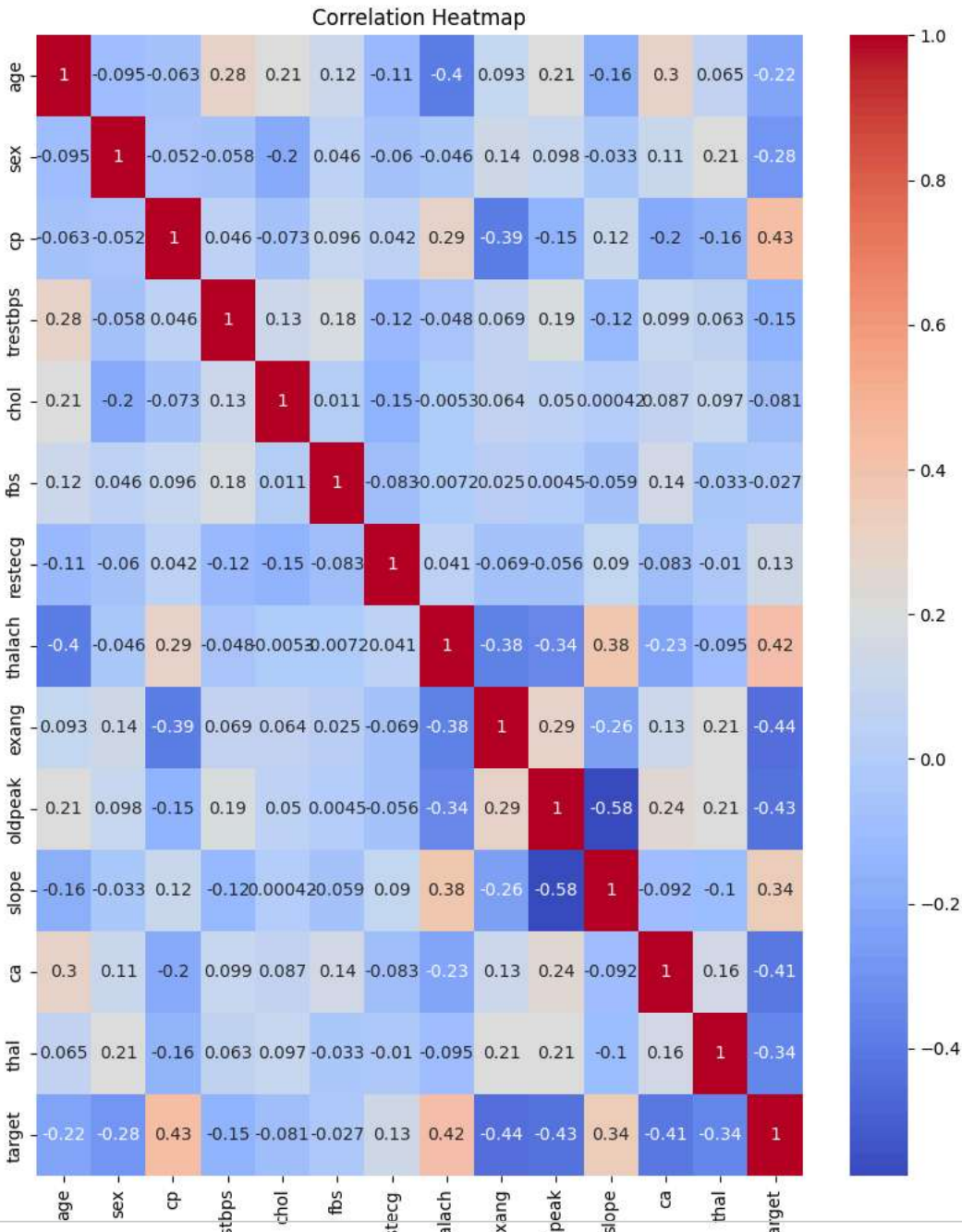
```
# Cholestoral Analysis
sns.histplot(df['chol'])
plt.title('Cholestrol Analysis')
plt.show()
```



```
# Gender Analysis
sns.countplot(x='sex', data=df)
plt.title('Gender Analysis')
plt.show()
```



```
# Correlation Heatmap
plt.figure(figsize=(10,12))
sns.heatmap(df.corr(),annot=True,cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



```
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
x= df.drop('target',axis=1)
y= df['target']
```

```
# split dataset into training and testing dataset
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

```
print(x_test)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
180	55	1	0	132	353	0	1	132	1	1.2	
229	64	1	2	125	309	0	1	131	1	1.8	
111	57	1	2	150	126	1	1	173	0	0.2	
247	66	1	1	160	246	0	1	120	1	0.0	
60	71	0	2	110	265	1	0	130	0	0.0	
..	...	...	..	...	...	...	...	...	...	...	
250	51	1	0	140	298	0	1	122	1	4.2	
104	50	1	2	129	196	0	1	163	0	0.0	
300	68	1	0	144	193	1	1	141	0	3.4	
194	60	1	2	140	185	0	0	155	0	3.0	
185	44	1	0	112	290	0	0	153	0	0.0	
	slope	ca	thal								
180	1	1	3								
229	1	0	3								
111	2	1	3								
247	1	3	1								
60	2	1	2								
..	...	..	...								
250	1	3	3								
104	2	0	2								

```
300      1      2      3
194      1      0      2
185      2      1      2

[61 rows x 13 columns]
```

```
print(y_train)

132      1
203      0
197      0
75       1
177      0
..
189      0
71       1
106      1
271      0
102      1
Name: target, Length: 241, dtype: int64
```

```
# create a logistic regression model
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
```

```
model.fit(x_train,y_train)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1): STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in: <https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
  ▾ LogisticRegression ⓘ ?
LogisticRegression()
```

```
y_prediction= model.predict(x_test)
print(y_prediction)

[0 0 1 0 1 1 1 0 0 0 1 0 1 0 1 1 1 0 0 0 1 0 0 1 1 1 0 1 0 1 0 0 1 0 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 0 0 0 0 1 1 0 0 0 1 0 1 1]
```

```
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test,y_prediction))

0.8524590163934426
```

Start coding or [generate](#) with AI.